Personalised Learning RAG and AI Mock Interview Assistant

Individual component report, SkillBridge (Academic-to-Industry Skill Bridge), project J26-SE-328-RP
Author: Dilmith Ranasinghe · **Date:** 17 October 2026 · **Code state:** `main` at `00edfe9` plus uncommitted work described in section 9

1. Summary

This component helps a student move from academic knowledge to industry readiness in two ways:

1. **Personalised Learning Assistant** (`services/learning-service`). The student asks a technical question; the service retrieves passages from a curated technical corpus and generates an answer grounded in those passages, with citations and a recommended next topic.
2. **AI Mock Interview Assistant** (`services/interview-service`). A video interviewer ("Elena") asks role-specific technical questions drawn from a rubric question bank, listens to the student's spoken answer, scores it on four criteria and gives spoken feedback.

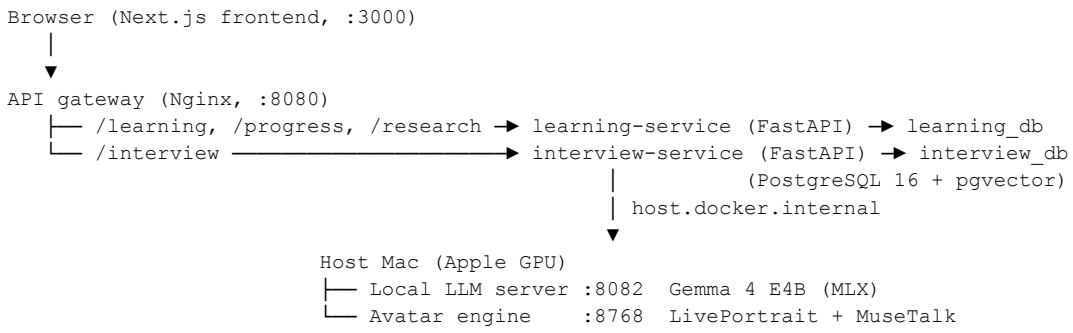
No machine learning model was trained or fine-tuned in this component. All models are pre-trained and used for inference. The research and engineering work is in the retrieval pipeline (hybrid retrieval, rank fusion, re-ranking, context compression), the interview workflow, the integration of several deep learning models into one real-time system, and the evaluation of local models for this task. Section 6 lists every model with its type.

The interview can now run **fully on a local machine with no cloud API key**: a local large language model on the Apple GPU, local speech recognition and local embeddings. Cloud models (Gemini, OpenAI) remain optional providers.

2. Scope and boundaries

Rule	How it is enforced
No skill-gap analysis	Student profile, target role and learning priority arrive pre-computed from other components as JSON contracts
Independent modules	Learning and interview are separate microservices with separate databases (`learning_db`, `interview_db`)
No automatic coupling	Interview results are stored in interview turn history only; they never change the learning syllabus
No skill tree	Progress is logged as topic mastery counts, without hierarchical tree structures

3. Architecture



- **Containers:** Docker Compose runs PostgreSQL, the services and the gateway.
- **Host Mac:** the LLM server and avatar engine run directly on the Mac, because they need the Apple GPU, which Docker cannot use.
- **Technology:** Python, FastAPI, SQLAlchemy (async), pgvector, Next.js, TypeScript, Tailwind CSS, Nginx, Docker, MLX, PyTorch.

4. Personalised Learning RAG (learning-service)

4.1 Knowledge base and ingestion

Item	Detail
Corpus	29 curated Markdown documents in four tracks: backend (10), database (6), DevOps (6), full-stack (7)
Chunking	Recursive character chunker, 600 characters with 100 overlap, aware of code fences and paragraphs
De-duplication	SHA-256 content hash per chunk, so re-ingestion is idempotent
Vector index	pgvector HNSW index (`m=16`, `ef_construction=64`, cosine distance)
Embeddings	`gemini-embedding-001`, 1536 dimensions (configurable: OpenAI, Gemini or mock)

4.2 Retrieval and generation pipeline

1. **Dense retrieval:** cosine similarity over pgvector embeddings.
2. **Sparse retrieval:** PostgreSQL full-text search ranked with `ts_rank_cd` (a BM25-style lexical ranking), with OR-query expansion.
3. **Hybrid fusion:** Reciprocal Rank Fusion, `score = w_d/(k + rank_dense) + w_s/(k + rank_sparse)`, with `k = 60` and equal weights (0.5 / 0.5), over a candidate pool of 3 × top-K.
4. **Second-stage re-ranking:** a rule-based scoring function, `0.45 × dense score + 0.30 × query-term coverage + exact-phrase bonus (0.25) + title/topic affinity (≤ 0.2)`, reducing about 10 candidates to the top 3. It is a hand-designed heuristic, not a neural cross-encoder.
5. **Context compression:** removes decorative Markdown, repeated blank lines and boilerplate openers, then truncates to 600 characters, reporting estimated token savings.
6. **Grounded generation:** the LLM (Gemini 2.5 Flash or OpenAI, configurable) must answer only from the retrieved passages and cite them as `[Source n: title]`. It also returns a recommended next topic.
7. **Telemetry and progress:** retrieval and generation latency and similarity scores go to `learning_experiment_logs`; topic progress per student goes to a separate table.

4.3 Research endpoints

`/research/retrieval-ablation` (dense vs sparse vs hybrid side by side), `/research/benchmark` (baseline direct-LLM vs RAG), `/research/benchmark-matrix`, `/research/golden-dataset` (40-item evaluation set), `/research/evaluate-query` and `/research/experiments`.

5. AI Mock Interview Assistant (interview-service)

5.1 Interview flow

1. **Question generation (RAG):** the service retrieves rubric questions for the target role from `interview_db` (pgvector, top-K = 3), skips topics already asked in the session, and asks the LLM to phrase one realistic, scenario-based question of at most 35 words. If the previous answer is supplied, the question probes the concepts that answer missed.
2. **Speech-to-text:** the browser records the answer (WebM/Opus or MP4) and the service transcribes it, with a technical-vocabulary prompt.
3. **Evaluation:** the LLM scores the answer from 1.0 to 5.0 on **technical accuracy, relevance, explanation quality and clarity**, and lists missing concepts, strengths and improvement areas. It also writes a short **spoken reaction** (at most 40 words) for the interviewer to say. Scores are validated (clamped to 1–5); a missing score is reported as an error rather than guessed.
4. **Report:** per-turn and average scores for the session.

5.2 Question bank

28 rubric questions with expected answers and grading criteria, across 4 roles (Backend 11, DevOps 6, Database 6, Full-Stack 5) and 3 levels (Entry 10, Mid 10, Senior 8). They are loaded automatically into `interview_db` with

embeddings on first start.

5.3 Configurable providers

Function	Providers	Default
Question writing and evaluation	`local` (host LLM), `gemini`, `openai`, `mock`	`local`
Embeddings	`local` (fastembed bge-small, 384-d), `gemini`, `openai`, `mock`	`local`
Speech-to-text	`local` (faster-whisper), `whisper_api`, `gemini`, `auto`	`local`
Interviewer video	`local` (avatar engine), `liveavatar` (HeyGen), `none`	`local`

5.4 Video interviewer integration

- **Render start:** rendering of the interviewer's spoken video starts as soon as a question or feedback text exists.
- **Sentence streaming:** text is split into sentence clips; the first sentence plays after about 11–18 s instead of waiting 30–60 s for one long clip. Clips are chained so the head motion continues between sentences.
- **Prefetching:** in hands-free mode, the next question is requested as soon as an answer is evaluated, so it renders while the interviewer gives feedback.
- **Cancellation:** clips nobody will watch are cancelled, so the current clip renders next.
- **Video proxy:** media is streamed through the service with HTTP Range support (needed for Safari).

5.5 Local avatar engine (separate folder: `Huggingbay/interview-avatar`)

Part	Role
LivePortrait	Animates the interviewer photo with head motion; a custom script (`animate.py`) adds natural blinking (2–6 s intervals, about 0.3 s per blink)
MuseTalk 1.5 (MLX port)	Lip-syncs the mouth region to the speech audio
TAESD decoder (ported to MLX)	Fast image decoder: about 8× faster decoding, about 2× faster lip-sync overall
macOS text-to-speech	The interviewer's voice
Job server (Flask)	One GPU job at a time, clip chaining, cancellation; GPU cache capped (memory fell from about 7 GB to about 2 GB)

6. Models used

Nothing was trained or fine-tuned. All models below are pre-trained by their publishers and used for inference only.

Model	Category	Architecture	Used for	Runs on
Gemma 4 E4B (Google), 4-bit MLX	Deep learning, LLM	Decoder-only transformer, instruction-tuned	Interview questions, evaluation, spoken feedback	Mac GPU (default)
Gemini 2.5 Flash (Google)	Deep learning, LLM	Transformer (cloud API)	Learning-RAG answers; optional interview provider	Cloud
GPT-4o-mini (OpenAI)	Deep learning, LLM	Transformer (cloud API)	Optional provider	Cloud
Phi-4-mini (Microsoft), 4-bit MLX	Deep learning, LLM	Decoder-only transformer	Evaluated as a lighter option; not selected	Mac GPU
bge-small-en-v1.5 (BAAI)	Deep learning, embedding	BERT-based bi-encoder, 384-d (ONNX)	Interview question retrieval	Container CPU

Model	Category	Architecture	Used for	Runs on
gemini-embedding-001 (Google)	Deep learning, embedding	Transformer encoder, 1536-d	Learning corpus retrieval	Cloud
Whisper small.en (OpenAI), faster-whisper int8	Deep learning, speech recognition	Encoder–decoder transformer	Transcribing spoken answers	Container CPU
LivePortrait (Kuaishou / KlingTeam)	Deep learning, generative vision	Implicit-keypoint motion, warping network and generator (CNN)	Interviewer head motion and blinking	Mac GPU (PyTorch MPS)
MuseTalk 1.5 (Tencent Music)	Deep learning, generative vision	Single-step latent inpainting UNet, SD VAE, Whisper-tiny audio encoder	Lip-sync	Mac GPU (MLX)
TAESD (madebyollin)	Deep learning, autoencoder	Small convolutional decoder	Fast latent-to-image decoding	Mac GPU (MLX)
InsightFace buffalo_l	Deep learning, vision	Face detector and 106-point landmark CNNs (ONNX)	Face location for cropping and blending	CPU

Non-neural techniques in the component

Technique	Category
PostgreSQL full-text ranking (`ts_rank_cd`)	Classical information retrieval (statistical, not ML)
Reciprocal Rank Fusion	Rank-aggregation algorithm
Second-stage re-ranker, context compression	Rule-based heuristics
RAG Triad metrics in `rag_evaluator.py`	Lexical heuristics (token overlap), not LLM-judged
HNSW vector index	Approximate nearest-neighbour search

What the model work consisted of

- **Integration and orchestration:** retrieval-augmented generation, prompt engineering and structured JSON output with validation.
- **Model selection by benchmark:** section 7.2.
- **Deployment engineering:** running pre-quantised 4-bit models; converting TAESD weights to MLX (a format conversion, not training); building a custom blink animation on LivePortrait's keypoint retargeting.

7. Testing and evaluation

7.1 Automated tests

Suite	Tests
`interview-service/tests/test_local_avatar.py`	25, all passing: providers, video proxy and seeking, validation, speech-to-text error handling, local LLM JSON handling, sentence streaming, cancellation
`interview-service/tests/test_e2e_interview_rag.py`, `test_interview_evaluation.py`	2 (need a live database)
`learning-service/tests/` (chunking, embeddings, end-to-end RAG)	6
Frontend	TypeScript type check and lint clean for the changed files

7.2 Local LLM benchmark (measured 17 Oct 2026, M4 MacBook Air, 16 GB)

Same interview prompts and two recorded answers (strong and weak), two runs each:

Model	Evaluations succeeded	Strong answer (tech / rel / expl / clarity)	Weak answer	Evaluation time	Peak memory
Gemma 4 E4B	**4 / 4**	4.5 / 5.0 / 4.0 / 4.5	**1.5 / 3.0 / 1.0 / 3.5**	about 10–12 s	5.4 GB
Phi-4-mini	2 / 4 (invalid JSON)	failed	1.5 / 5.0 / 1.5 / 5.0	about 3–6 s	3.0 GB

Gemma 4 E4B was selected because it scored fairly and reliably. Phi-4-mini rated an answer of "just make the pool bigger" 5/5 for relevance.

7.3 End-to-end timings (local stack, measured)

Step	Time
Question text generated	about 2–11 s
Interviewer starts speaking (first sentence clip)	about 11–18 s
Speech-to-text (local Whisper, about 20 s of audio)	about 6 s
Evaluation (Gemma 4 E4B)	about 12 s

7.4 Known issue: research benchmark figures in code are not measured

`learning-service/app/services/rag_evaluator.py` and the frontend's Viva Inspector contain values that are **fixed in code, not measured**:

- **Benchmark matrix:** `run_full_benchmark_matrix()` returns fixed metric values (for example faithfulness 0.4820 / 0.7640 / 0.9680), a fixed **p-value of 0.0018**, and a pre-written conclusion ("+26.7% over Naive RAG …"). `VivaInspectorView.tsx` shows the same numbers as a "Publication Empirical Benchmark".
- **Per-item evaluation:**
  - The baseline's faithfulness is fixed at 0.42.
  - Naive RAG relevance is set to the baseline's plus 0.15, and its latency is padded by 120 ms.
  - Naive RAG faithfulness is computed on the expert reference answer, not a generated answer.
  - The advanced pipeline's scores are floored (faithfulness at least 0.92, relevance 0.90, precision 0.88, recall 0.94).
- **Earlier report:** the October 5 report shows a generation latency of 0.00 ms, meaning no LLM was called in those runs (the template fallback answered).

These figures must not be reported as experimental results. Before the research evaluation is written up, the floors and fixed values should be removed, the 40-item golden set should be run end to end, and real statistics (for example a paired test across items) should be computed. An LLM-as-judge or embedding-based faithfulness check would make the triad metrics more defensible than token overlap.

8. Engineering improvements made

- **No fabricated data in the interview:** failed transcriptions, blocked microphones and failed evaluations now show errors instead of inserting invented answers or scores. The hard-coded 86% eye-contact figure was removed.
- **No silent random vectors:** embeddings no longer fall back to random vectors when a provider fails (that silently produced random retrieval).
- **Startup bug fixed:** a broken import in `interview-service/app/main.py` had prevented the question bank from loading automatically.
- **Robustness:** invalid IDs return 404 instead of 500; speech-to-text works across browsers (WebM and MP4 converted with ffmpeg).
- **Real-interview flow:** the candidate camera now displays correctly; hands-free mode (question, automatic recording, evaluation, spoken feedback, next question); follow-up questions that probe missed concepts; no repeated topics within a session.

- **Performance:** sentence-streamed interviewer video, prefetching, clip cancellation; avatar engine memory reduced from about 7 GB to about 2 GB; LLM "thinking mode" disabled (it made Gemma 10× slower and truncated its answers).

9. Work status

- **Committed on main:** the advanced RAG pipeline, multi-turn interview memory, the avatar streaming services and proxy, and the frontend integration (commits `aac1df8` to `00edfe9`).
- **Uncommitted, at the time of writing:**
  - local LLM, embedding and Whisper providers;
  - Gemini speech-to-text option;
  - spoken feedback and hands-free flow;
  - camera fix;
  - removal of fabricated fallbacks;
  - sentence streaming, prefetching and cancellation;
  - the startup bug fix;
  - 25 tests.
- **Local-only environment fix:** a Docker Compose override (`~/.skillbridge-local/docker-compose.override.yml`) works around the external drive's `noowners` mount, which stops PostgreSQL from running its init script.

10. Limitations and future work

1. **Benchmark:** replace the fixed benchmark values with a real run over the golden set (section 7.4).
2. **Local learning service:** the learning service still uses Gemini for generation and embeddings. Moving it to the local providers needs re-embedding the corpus at 384 dimensions.
3. **Neural re-ranker:** replace the heuristic re-ranker with a cross-encoder (for example `bge-reranker-base`) and measure the difference.
4. **Interviewer realism:** a separate "listening" idle clip, a neural text-to-speech voice (for example Kokoro-82M), and gaze and expression variation.
5. **Speed:** on the 8-core M4 GPU, rendering is about 2.5× slower than speech, so pauses between sentences can occur. A GPU server (24 GB) or an M4 Pro would allow seamless real-time speech.
6. **Question bank:** extend beyond 28 questions and four roles.

11. How to run (local, no API key)

```
# Host Mac (Huggingbay folder)
./start-avatar.sh          # avatar engine, port 8768
./start-llm.sh             # Gemma 4 E4B, port 8082

# Project folder (J26-SE-328-RP)
docker compose -f infrastructure/docker-compose.yml \
  -f ~/.skillbridge-local/docker-compose.override.yml up -d
cd frontend && npm run dev # http://localhost:3000/mock-interview
```